

Università degli Studi di Urbino Carlo Bo
Corsi di Laurea in Informatica Applicata
Programmazione e Modellazione a Oggetti, a.a. 2025/2026

Relazione per il Progetto del Corso di
Programmazione e Modellazione a Oggetti

Overdrive Run

Autori

Alessandro Catalano – matricola 334827
Alessio Cavalieri – matricola 336599

Sessione d'esame 2025/2026

Indice

1	Analisi	2
1.1	Requisiti	2
1.2	Modello del dominio	3
2	Design	5
2.1	Architettura	5
2.1.1	Strategia di gestione degli errori	6
2.2	Design dettagliato	7
2.2.1	Cavalieri Alessio	7
2.2.2	Catalano Alessandro	12
3	Sviluppo	16
3.1	Testing automatizzato	16
3.2	Metodologia di lavoro	17
3.2.1	Alessio Cavalieri	18
3.2.2	Alessandro Catalano	18
3.2.3	Componenti sviluppate in collaborazione	18
3.3	Note di sviluppo	19
3.3.1	Alessio Cavalieri	19
3.3.2	Alessandro Catalano	19
	Conclusioni	21

Capitolo 1

Analisi

Il progetto realizza un videogioco platform a scorrimento orizzontale in stile *runner*. Il giocatore controlla un cubo che avanza a velocità costante e deve raggiungere la fine del livello superando una sequenza di ostacoli con salti temporizzati: il contatto con un ostacolo letale termina immediatamente il tentativo e obbliga a ricominciare il livello da capo.

1.1 Requisiti

Funzionalità obbligatorie

- Due livelli preimpostati, selezionabili da un menu iniziale. Ciascun livello propone un percorso differente e una difficoltà crescente, con ostacoli disposti per mettere alla prova riflessi e coordinazione.
- Game loop strutturato nelle tre fasi canoniche: gestione dell'input (pressione della barra spaziatrice), aggiornamento dello stato (scorrimento degli ostacoli, fisica del salto, rilevamento delle collisioni) e rendering della scena.
- Hitbox coerenti e controlli di collisione affidabili fra cubo e entità.
- Salto con la barra spaziatrice o, in alternativa, con la freccia verso l'alto. Il cubo mantiene una velocità di scorrimento costante per tutta la partita; tenendo premuto il tasto salta di nuovo non appena torna a terra, sfruttando la ripetizione automatica dei tasti fornita dal sistema operativo.
- Entità aggiuntive che alterano la traiettoria verticale, permettendo salti più alti o incatenati.
- Pausa della partita, con possibilità di riprendere o di tornare al menu principale.

Funzionalità opzionali

- Colonna sonora dedicata per il menu e per ciascun livello, più effetti sonori sull'esito del tentativo.
- HUD di gioco con nome del livello, contatore dei tentativi, barra di avanzamento con percentuale di completamento e promemoria dei comandi.
- *Jump buffering*: un input di salto ricevuto poco prima dell'atterraggio resta valido e viene applicato al contatto con il terreno, così che una pressione leggermente anticipata non venga persa.
- Schermata di fine tentativo che distingue vittoria e sconfitta e riepiloga tentativi e percentuale raggiunta.

Diagramma dei casi d'uso

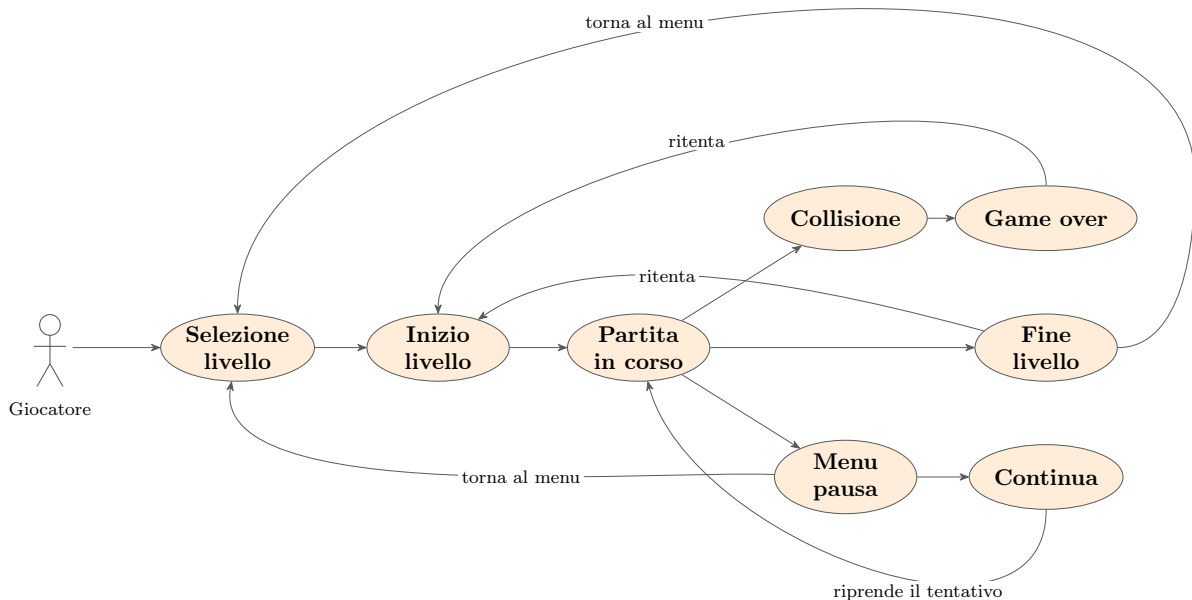


Figura 1.1: Casi d'uso del giocatore.

Il giocatore parte dalla selezione del livello e avvia un tentativo. Da lì la partita in corso può concludersi in due modi: con una collisione letale, che porta alla schermata di game over, oppure con il raggiungimento del traguardo. In entrambi i casi il tentativo successivo riparte dall'inizio dello stesso livello, oppure si può tornare al menu. Durante la partita il giocatore può inoltre mettere in pausa: scegliendo di continuare il tentativo riprende dal punto in cui era stato sospeso, senza ricominciare, mentre abbandonando si torna al menu.

1.2 Modello del dominio

Descriviamo qui il dominio del problema, indipendentemente da come il software sarà poi organizzato: le entità in gioco, le loro proprietà e le relazioni che le legano. Lo schema che segue è di analisi, quindi riporta concetti e non classi: la struttura interna dell'applicazione è discussa nel capitolo di design.

Le entità che compongono il dominio sono le seguenti.

- **Il cubo** è l'unico personaggio controllato dal giocatore. Possiede una posizione nello spazio di gioco e un'area di collisione che determina le interazioni con gli altri elementi del livello. La sua posizione verticale varia nel tempo per effetto della gravità e del salto; quella orizzontale è fissa rispetto allo schermo, perché è il livello a scorrere.
- **Il livello** definisce il contesto di gioco: è caratterizzato da un nome, dalla quota del terreno, da una traccia musicale associata e dall'insieme delle entità che lo popolano.

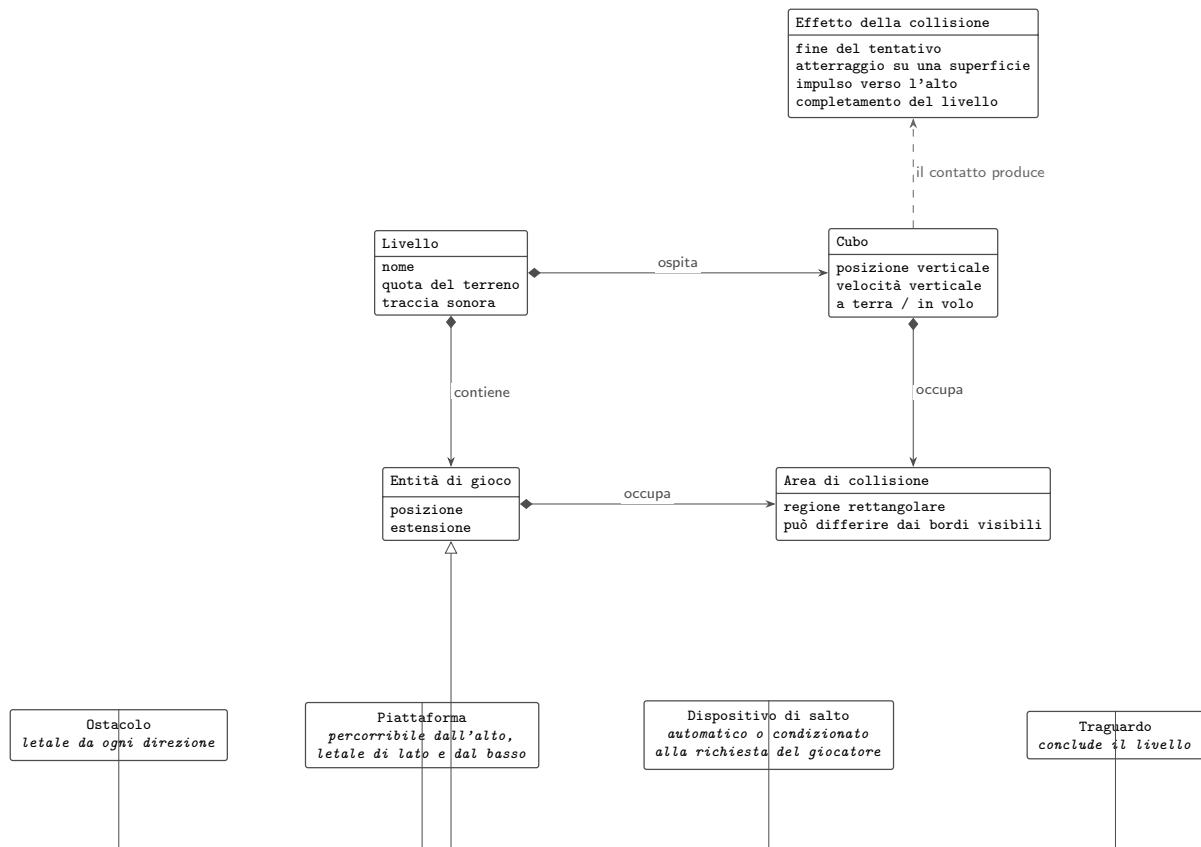


Figura 1.2: Modello del dominio: concetti del problema e loro relazioni.

- **Le entità di gioco** sono il concetto astratto che raggruppa tutti gli oggetti presenti nel livello. Ogni entità occupa una posizione, possiede un'area di collisione e, al contatto con il cubo, produce un effetto sullo stato della partita. Comprende:
 - *l'ostacolo*, entità letale: il contatto da qualunque direzione termina immediatamente il tentativo;
 - *la piattaforma*, superficie solida formata da uno o più blocchi su cui il cubo può atterrare. Il contatto dall'alto la rende percorribile, il contatto laterale e quello dal basso sono letali;
 - *i dispositivi di salto*, che alterano la traiettoria verticale del cubo. Se ne distinguono due varianti: una che agisce automaticamente al contatto e una che richiede che sia pendente una richiesta di salto nel momento del contatto;
 - *il traguardo*, marker posto al termine del livello, il cui raggiungimento chiude la partita con esito positivo.
- **L'area di collisione** definisce la regione rettangolare di ciascuna entità rilevante ai fini delle interazioni fisiche. Può differire dai bordi visivi dell'entità per garantire un'esperienza di gioco equa.
- **L'effetto** descrive la conseguenza di una collisione: la partita può terminare con una sconfitta, il cubo può atterrare su una superficie, ricevere un impulso verso l'alto, oppure il livello può concludersi con successo.

Capitolo 2

Design

2.1 Architettura

Il software è organizzato secondo il pattern architetturale Model–View–Controller. La scelta risponde all'esigenza di separare la logica di gioco dalla sua rappresentazione visiva, così che le due componenti siano modificabili e collaudabili in modo indipendente.

Il Controller funge da disaccoppiatore fra Model e View: i pannelli di menu comunicano con esso esclusivamente tramite **ActionListener** e comandi tipizzati, e il **GameRenderer** legge i dati di gioco attraverso i metodi pubblici di **GameController**, senza invocare mai il **GameEngine**.

La separazione è verificabile guardando le dipendenze fra i package: nessuna classe del package **model** importa qualcosa da **view**, **controller** o **service**. Il modello conosce solo la libreria standard e il parser JSON, ed è quindi collaudabile e riusabile senza alcuna interfaccia grafica.

Nell'altra direzione il disaccoppiamento non è completo, ed è corretto dirlo. La View usa alcuni tipi del Model — **Cube**, **Entity** e **GameConstants** nel renderer, **GameEngine** nel frame principale, che è il punto di ingresso dell'applicazione — pur ottenendo sempre i valori attraverso il Controller. Il Controller, a sua volta, dichiara i campi **OverdriveRun** e **GameRenderer** con il loro tipo concreto: una sostituzione integrale della View richiederebbe quindi di ritoccare quei due riferimenti. Rimuovere del tutto il vincolo significherebbe introdurre due interfacce, una per il contenitore di pannelli e una per la superficie di disegno; è un passo che non abbiamo fatto perché con una sola View il costo in indizione non sarebbe ripagato, ma è la modifica da fare se la View dovesse essere sostituita davvero.

Un secondo compromesso riguarda la direzione opposta. L'interfaccia **Entity** espone, oltre alla geometria e all'effetto della collisione, anche **getWidth()**, **getHeight()**, **getNumber()** e soprattutto **getSpritePath()**: quest'ultimo colloca nel Model il percorso di una risorsa grafica, che è a tutti gli effetti un dato della View. La scelta è deliberata: con cinque sole entità, l'alternativa — una mappa da tipo di entità a sprite mantenuta nel renderer, oppure un enum di tipi da tradurre in percorso — avrebbe aggiunto un punto da aggiornare a ogni nuova entità, cioè esattamente ciò che il resto del design cerca di evitare. Resta il fatto che il Model non è puro quanto le sue dipendenze suggeriscono: non importa nulla dalla View, ma ne conosce una convenzione.

Punti d'ingresso e ruoli. Ciascuno dei tre componenti architetturali ha un punto d'ingresso unico, cioè il tipo attraverso il quale il resto del sistema lo raggiunge.

- **Model** — punto d'ingresso **GameEngine**. Ricopre due ruoli: custodisce lo stato del tentativo in corso (cubo, entità, esito, avanzamento) e lo fa evolvere di un tick alla volta applicando fisica, scorrimento e collisioni. Riceve comandi (**update()**, **requestJump()**, **resetGame()**, **loadLevel()**, **incrementAttempts()**) e risponde a interrogazioni; non notifica nessuno e non

conosce l'esistenza degli altri componenti. L'interfaccia **Entity** è il secondo tipo pubblico del modello, ed è il punto di estensione con cui si aggiungono nuovi elementi di gioco.

- **View** — punto d'ingresso **OverdriveRun**, il contenitore che possiede le schermate e le alterna. Ricopre due ruoli: presentare lo stato di gioco, delegato a **GameRenderer**, e raccogliere le azioni dell'utente, delegate ai pannelli di menu. Non decide nulla: ogni evento raccolto viene inoltrato al Controller.
- **Controller** — punto d'ingresso **GameController**. Ricopre tre ruoli: possiede il ciclo di gioco e il thread su cui gira, traduce le azioni dell'utente in comandi per il Model (delegando i menu a **MenuController** e la tastiera di gioco a **LevelController**), e serve alla View i dati del frame corrente attraverso i propri accessori.

Lo scambio di informazioni segue quindi un percorso unico: la View genera eventi verso il Controller, il Controller invia comandi al Model e ne interroga lo stato, e la View rilegge quello stato passando di nuovo dal Controller. Non esiste alcun canale diretto fra View e Model, né alcuna notifica dal Model verso l'alto.

Il package **service** contiene **MusicPlayer**, un componente trasversale che gestisce musica di sottofondo ed effetti sonori. Non appartiene a nessuno dei tre livelli MVC: è usato dal Controller, per avviare e fermare la musica in base allo stato della partita, e dal frame principale, per la traccia del menu iniziale. Il Model non lo usa affatto: si limita a esporre, come dato del livello, il percorso della traccia da riprodurre.

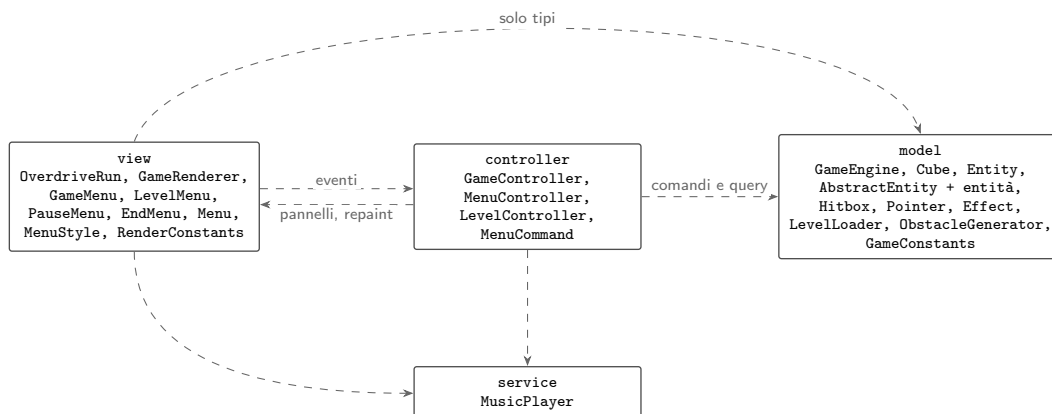


Figura 2.1: Dipendenze fra i package. Il modello è l'unico senza dipendenze uscenti.

2.1.1 Strategia di gestione degli errori

La filosofia adottata è di degradare invece di terminare, con una sola eccezione consapevole. Un errore che rende il gioco ingiocabile deve essere fatale e visibile; ogni altro errore deve essere segnalato allo sviluppatore e assorbito, perché una partita che continua con un asset mancante è preferibile a una finestra che si chiude.

- **Livello iniziale non caricabile.** È l'unico caso fatale: il costruttore di **GameEngine** propaga l'**IOException** invece di assorbirla, perché senza un livello non esiste alcuno stato giocabile su cui ripiegare. Il **main** intercetta l'eccezione, mostra un dialogo di errore e termina.
- **Cambio di livello fallito.** Quando **loadLevel()** è invocato a partita già avviata, il fallimento viene segnalato e il livello corrente resta caricato: il gioco continua a essere giocabile.

- **Segmento JSON malformato.** `getMap()` racchiude la creazione di ogni singolo segmento in un blocco protetto: un segmento invalido viene segnalato con il proprio indice e saltato, senza compromettere il resto del livello.
- **Classe di entità sconosciuta o senza il costruttore richiesto.** `ObstacleGenerator` distingue i due casi con eccezioni e messaggi diversi, che indicano se correggere il JSON o la classe.
- **Sprite mancante o non decodificabile.** Sostituita da un placeholder magenta, come descritto nella sezione sul rendering.
- **Risorsa audio mancante.** Segnalata e ignorata: la partita prosegue senza musica.
- **Comando di menu non riconosciuto.** Segnalato e ignorato, senza effetti sullo stato dell'applicazione.

2.2 Design dettagliato

Ciascuna delle due sottosezioni che seguono è di responsabilità esclusiva del membro del gruppo che le dà il nome. Per ogni aspetto approfondito sono riportati il problema affrontato, le alternative valutate con i rispettivi pro e contro, la soluzione adottata con il relativo schema UML e, dove pertinente, il pattern di progettazione impiegato e il modo in cui è reificato nel codice.

2.2.1 Cavalieri Alessio

2.2.1.1 Gerarchia delle entità di gioco e gestione delle collisioni

Problema. Il livello è composto da entità di tipo diverso: ostacoli, piattaforme, dispositivi di salto e traguardo che condividono alcune caratteristiche comuni (posizione, area di collisione, capacità di scorrere sullo schermo) ma si differenziano nel comportamento al contatto con il cubo. Era necessario modellare questa famiglia di oggetti evitando la duplicazione di codice e permettendo al motore di trattare tutte le entità in modo uniforme, senza conoscere il tipo specifico di ciascuna.

Alternative considerate. La prima ipotesi è stata una singola classe `Ostacolo` con un campo *tipo* e uno `switch` sul tipo dentro il motore. È più immediata da scrivere e tiene tutto il comportamento in un punto solo, ma concentra nel motore la conoscenza di ogni entità: aggiungerne una richiede di modificare il motore, e lo `switch` tende a essere replicato ovunque serva distinguere i casi. La seconda ipotesi era una gerarchia senza classe astratta intermedia, con ogni entità che implementa direttamente l'interfaccia: evita un livello di ereditarietà ma costringe a ripetere in ogni classe lo stesso codice di scorrimento e gli stessi accessori. Abbiamo scelto la terza via, interfaccia più classe astratta: costa un livello di gerarchia in più, e obbliga a passare per un metodo astratto anche quando la hitbox coincide con lo sprite, ma è l'unica che tiene insieme codice condiviso e comportamento variabile senza far crescere il motore.

Soluzione. Il problema è stato risolto combinando polimorfismo e un metodo factory astratto. La classe base concentra il codice invariante, mentre i due punti di variazione sono delegati alle sottoclassi: `createHitbox()`, invocato una sola volta dal costruttore di `AbstractEntity` per costruire l'area di collisione, e `onCollision()`, invocato a ogni contatto. Preferiamo non presentarlo come *Template Method*: `scroll()` è un metodo concreto senza passi variabili, e `createHitbox()` è un hook di costruzione, non uno stadio di un algoritmo definito nella superclasse.

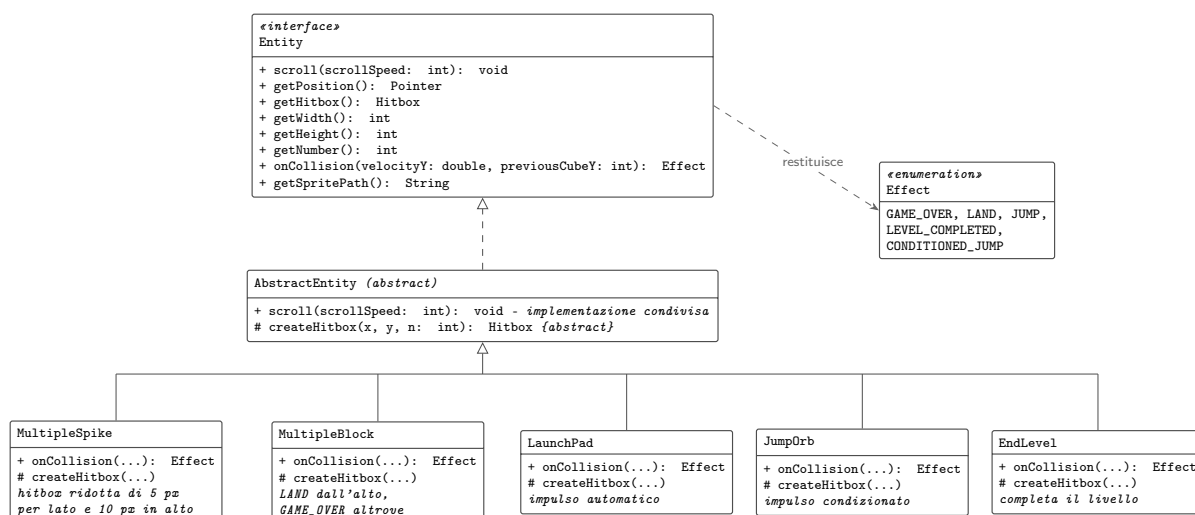


Figura 2.2: Gerarchia delle entità di gioco.

L'interfaccia `Entity` definisce il contratto comune a tutte le entità: `scroll()`, `onCollision()`, `getSpritePath()` e gli accessori geometrici. La classe astratta `AbstractEntity` fornisce l'implementazione condivisa di `scroll()`, che aggiorna con un unico delta sia la posizione visiva (`Pointer`) sia l'area di collisione (`Hitbox`); dichiara invece astratto il metodo `createHitbox()`, che ogni sottoclasse implementa per definire la propria area di collisione, eventualmente ridotta rispetto ai bordi dello sprite come avviene per `MultipleSpike`. Poiché la chiamata avviene dal costruttore della classe base, le implementazioni possono basarsi soltanto sui propri parametri e su costanti statiche, mai su campi della sottoclasse, che a quel punto non sono ancora inizializzati: è un vincolo documentato nel Javadoc del metodo.

Il metodo `onCollision()` è il punto di variazione della gerarchia: ogni sottoclasse restituisce un valore dell'enum `Effect` che il motore interpreta nel proprio ciclo di aggiornamento senza conoscere il tipo dell'entità. Il design permette di aggiungere un nuovo tipo di entità creando una classe nel package `model` e referenziandola nel JSON del livello, senza modificare il motore.

La scelta di un enum tipizzato per gli effetti, invece di costanti intere o di stringhe, rende lo `switch` del motore verificabile staticamente dal compilatore: un valore nuovo o rinominato viene segnalato in compilazione.

L'unica entità con un comportamento non banale è `MultipleBlock`, che deve distinguere un atterraggio da un impatto laterale:

```

public Effect onCollision(double velocityY, int previousCubeY) {
    int previousCubeBottom = previousCubeY + GameConstants.SIZE;
    int blockTop           = getHitbox().getY();

    boolean fallingDown = velocityY >= 0;
    boolean wasAbove     = previousCubeBottom
                           <= blockTop + GameConstants.PLATFORM_SAFE_ZONE;

    return (fallingDown && wasAbove) ? Effect.LAND : Effect.GAME_OVER;
}
  
```

La tolleranza `PLATFORM_SAFE_ZONE`, pari a 20 pixel, è necessaria perché il cubo si muove a passi discreti: con una velocità verticale che può superare i dieci pixel per tick, alla fine di un tick il cubo può trovarsi già qualche pixel dentro la superficie su cui stava legittimamente atterrando. Senza la tolleranza quel frame verrebbe letto come un impatto laterale e ucciderebbe il giocatore. Il valore è stato tarato sperimentalmente: sotto i 10 pixel circa vengono rifiutati atterraggi legittimi, sopra i 30 il cubo sopravvive a urti chiaramente laterali.

Il completamento del livello è l'unico effetto che non viene applicato subito: `isLevelCompleted` viene tradotto in `isGameOver` solo alla fine del tick. In questo modo il Controller rileva la fine del tentativo con un unico controllo su `isGameOver()` e interroga poi `isLevelCompleted()` per sapere se è stata una vittoria o una morte.

2.2.1.2 Caricamento dei livelli e generazione riflessiva delle entità

Problema. Il contenuto di ciascun livello, tipo e posizione di ogni entità è definito in un file JSON. Era necessario leggere questa configurazione e istanziare le entità corrispondenti senza dover aggiornare il codice del motore ogni volta che si aggiunge un nuovo tipo di entità. Una soluzione basata su uno `switch` fisso avrebbe violato il principio Open/Closed.

Formato del file di livello. Ogni livello è un oggetto JSON con quattro campi, di cui `music` è opzionale:

```
{
  "level_name": "ODR_Lvl_1",
  "ground_y": 400,
  "music": "/music/Dash-Orbit.wav",
  "segments": [
    { "class": "MultipleSpike", "x": 26, "y": 1, "n": 3 },
    { "class": "MultipleBlock", "x": 57, "y": 1, "n": 3 },
    { "class": "LaunchPad", "x": 66, "y": 0.25, "n": 1 },
    { "class": "EndLevel", "x": 162, "y": 7, "n": 1 }
  ]
}
```

Il campo `class` è il nome semplice di una classe del package `model`, `n` è il numero di ripetizioni orizzontali dello sprite e `x` e `y` sono coordinate logiche espresse in tile, convertite in pixel secondo

$$x_{px} = x \cdot \text{SIZE} \quad y_{px} = \text{ground_y} - y \cdot \text{SIZE}$$

in modo che nel file di livello l'asse verticale cresca verso l'alto, come è naturale quando si disegna un livello a mano. La coordinata `y` è letta come numero in virgola mobile: valori frazionari servono per le entità basse, come il `LaunchPad` dell'esempio, che a quota 0,25 risulta appoggiato al terreno. Il campo `music` è opzionale: quando manca, `LevelLoader` ripiega su una traccia di riserva, così che un livello scritto a mano resti riproducibile anche senza audio dedicato.

Alternative considerate. L'alternativa naturale è una factory con uno `switch` sul nome della classe. È più semplice da leggere, non usa reflection e segnala in compilazione un nome di classe scritto male; in cambio va modificata a ogni nuova entità, che è esattamente ciò che volevamo evitare. Abbiamo valutato anche di registrare le entità in una mappa da nome a costruttore, popolata a mano all'avvio: elimina la reflection sulla risoluzione della classe ma non quella sull'invocazione, e sposta soltanto il punto da aggiornare. La factory riflessiva ha il costo di spostare a runtime un errore che uno `switch` coglierebbe in compilazione, e per questo il messaggio d'errore indica esplicitamente il file e il segmento da correggere; in cambio nessuno dei due file coinvolti va toccato quando si aggiunge un'entità.

Soluzione. Il caricamento è diviso in due componenti con responsabilità separate. `LevelLoader` si occupa dell'I/O e del parsing: legge il file dalla classpath, ne estrae i quattro campi e li memorizza. Il metodo `getMap()` viene invocato a ogni reset della partita e costruisce ogni volta una lista fresca di entità, garantendo che le istanze siano indipendenti fra un tentativo e il successivo.

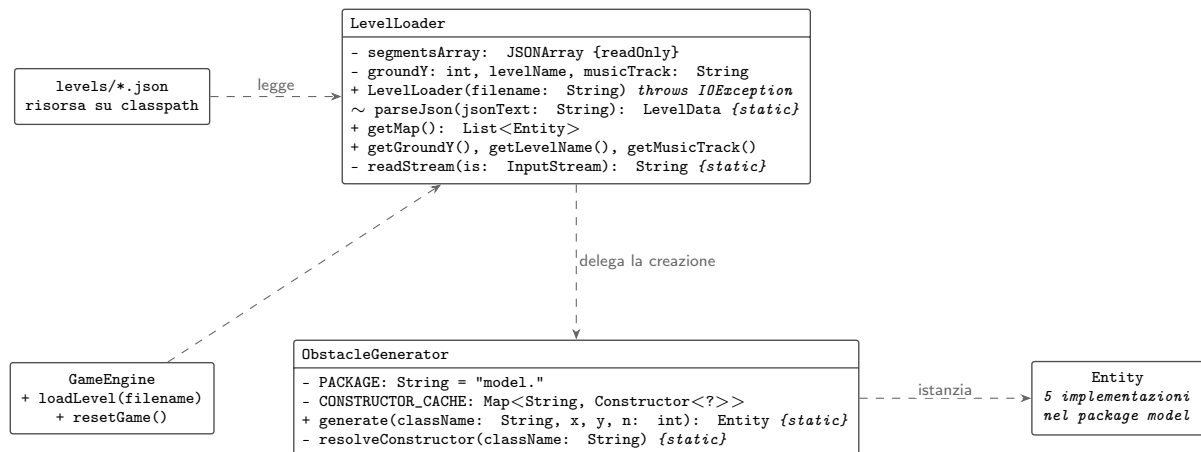


Figura 2.3: Caricamento di un livello e creazione riflessiva delle entità.

La creazione delle singole entità è delegata a **ObstacleGenerator**, una factory riflessiva. Dato il nome della classe come stringa, il metodo **generate()** risolve la classe nel package **model** tramite **Class.forName()**, verifica che implementi **Entity**, individua il costruttore con firma **(int, int, int)** e lo invoca. L'approccio rende il sistema aperto all'estensione: basta creare una nuova classe nel package **model** con il costruttore corretto e referenziarla nel JSON.

I costruttori risolti sono memorizzati in una **ConcurrentHashMap**: **generate()** viene chiamato una volta per segmento a ogni caricamento e a ogni nuovo tentativo, quindi senza la cache la stessa ricerca riflessiva verrebbe ripetuta centinaia di volte per partita. La cache separa inoltre la risoluzione del costruttore, che può fallire per un errore nel JSON, dalla sua invocazione, che può fallire solo per un difetto del codice: i due casi lanciano eccezioni diverse e con messaggi diversi.

La separazione fra I/O e parsing è comunque netta: **parseJson()** accetta direttamente una stringa e non conosce né il file system né la classpath. Va però detto che questa separazione non è stata sfruttata dai test: il metodo è dichiarato package-private nel package **model**, mentre le classi di test risiedono nel package **test** e non possono quindi raggiungerlo. Il caricamento dei livelli resta di conseguenza privo di collaudo automatico, come riportato fra i limiti noti.

2.2.1.3 Ciclo di gioco a timestep fisso e gestione della concorrenza

Problema. Il gioco deve aggiornarsi a 60 Hz in modo stabile, indipendentemente dal tempo impiegato per disegnare un frame. Gli input del giocatore arrivano sul thread Swing (EDT), mentre la logica di gioco gira su un thread dedicato: era necessario garantire la consistenza dello stato condiviso senza introdurre deadlock o rallentamenti percepibili.

Alternative considerate. La soluzione più semplice sarebbe stata un **javax.swing.Timer**, che esegue il tick direttamente sull'EDT: elimina ogni problema di concorrenza, perché input, logica e disegno girano sullo stesso thread, ma fa dipendere la frequenza di aggiornamento dal tempo di disegno, e un frame lento rallenta la partita invece di essere recuperato. L'altra alternativa era un loop a passo variabile, che integra la fisica moltiplicando per il tempo trascorso: si adatta a qualunque hardware, ma rende il gioco non riproducibile, e in un platform di precisione un salto che riesce su una macchina e fallisce su un'altra è un difetto grave — oltre a rendere impossibili i test a valori esatti descritti nella sezione 3.1. Il timestep fisso su thread dedicato ci ha costretti a gestire esplicitamente la sincronizzazione, ma è l'unico che garantisce una fisica deterministica.

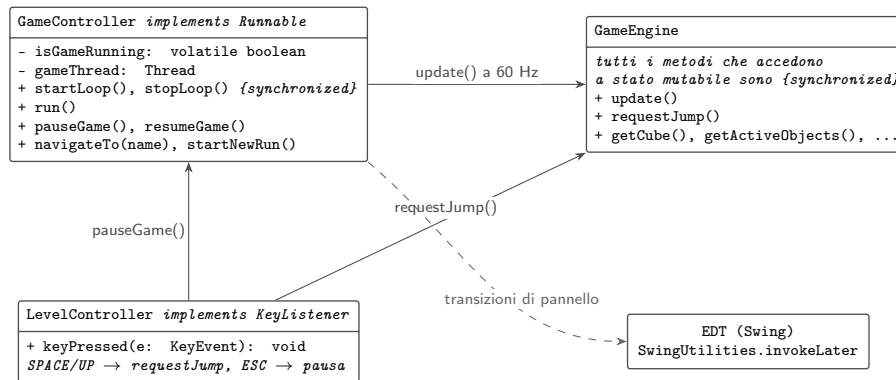


Figura 2.4: Ciclo di gioco e interazione fra thread.

Soluzione. Il ciclo di gioco è implementato in **GameController**, che implementa **Runnable** ed è eseguito su un thread demone dedicato. Il modello adottato è un fixed-timestep loop con catch-up:

```

long nextTick = System.nanoTime() + TICK_NANOS;
while (isGameRunning) {
    while (System.nanoTime() >= nextTick) {
        gameEngine.update();
        nextTick += TICK_NANOS;
        ...
    }
    gameRenderer.repaint();
    long timeUntilNext = nextTick - System.nanoTime();
    if (timeUntilNext > MIN_SLEEP_NANOS) {
        Thread.sleep((timeUntilNext - SLEEP_MARGIN_NANOS) / 1_000_000L);
    }
}

```

Il dettaglio che rende il ciclo stabile è che **nextTick** viene incrementato di esattamente un tick alla volta invece di essere ricalcolato dall'istante corrente: in questo modo l'orologio logico resta ancorato al tempo reale e un frame che ha impiegato troppo viene compensato eseguendo più aggiornamenti consecutivi, non rallentando silenziosamente il gioco. Il rendering avviene una volta per frame, indipendentemente da quanti aggiornamenti siano stati eseguiti. Dopo ogni frame il thread dorme per il tempo residuo meno un margine di 1 ms, ed evita di dormire del tutto quando mancano meno di 2 ms, perché la risoluzione di **Thread.sleep** è dell'ordine del millisecondo e una pausa più breve supererebbe il tick successivo. In questo modo si evita il busy-waiting senza perdere precisione.

La thread-safety è gestita su due livelli. Tutti i metodi di **GameEngine** che leggono o modificano stato mutabile sono dichiarati **synchronized**, compresi gli accessori dei metadati del livello: la richiesta di salto, che può arrivare dall'EDT in qualsiasi momento, entra quindi nel monitor dell'oggetto come ogni altra operazione. Il flag **isGameRunning** del controller è invece **volatile**, perché viene scritto da un thread e letto dall'altro fuori da ogni lock.

Le transizioni di pannello decise dal thread di gioco, come la pausa o il game over, sono schedate sull'EDT tramite **SwingUtilities.invokeLater()**, rispettando il threading model di Swing. I metodi **stopLoop()** e **pauseGame()** sono **synchronized** e attendono la terminazione del thread con una **join** di al più 300 ms: questo impedisce che un nuovo tentativo inizi prima che il ciclo precedente si sia concluso. Anche la pausa passa da **stopLoop()** anziché limitarsi ad azzerare il flag, perché una ripresa immediata potrebbe altrimenti avviare un secondo ciclo mentre il precedente è ancora in attesa, facendo avanzare il motore due volte per tick. Lo stesso rigore è applicato a **MusicPlayer**, i cui metodi che mutano il campo condiviso **clip** sono **synchronized** perché invocati da thread diversi.

2.2.2 Catalano Alessandro

2.2.2.1 Navigazione tra schermate e pattern Command nei menu

Problema. L'applicazione presenta cinque pannelli distinti: menu principale, selezione livello, gioco, pausa e schermata finale tra i quali occorre navigare in risposta alle azioni dell'utente. Era necessario che i pulsanti dei menu potessero comunicare azioni al Controller senza usare stringhe letterali, soggette a errori tipografici non rilevabili dal compilatore, e senza che i pannelli della View conoscessero direttamente i metodi del Controller.

Alternative considerate. Si poteva creare e distruggere una finestra per ogni schermata, ma ogni transizione avrebbe fatto sparire e ricomparire la finestra, e lo stato della partita in pausa sarebbe andato perso. Per i comandi, l'alternativa più diffusa è che ogni pulsante registri una propria lambda che chiama direttamente il metodo del Controller: è la soluzione più breve, ma lega ciascun pannello all'API del Controller, e sostituire la View comporterebbe riscrivere quelle chiamate. Una terza possibilità era usare le stringhe letterali come action command, scartata subito perché un errore di battitura diventa un comando silenziosamente ignorato. L'enum costa una conversione da stringa a valore tipizzato a ogni evento, ma lascia i pannelli all'oscuro dei metodi del Controller e sposta gli errori in compilazione.

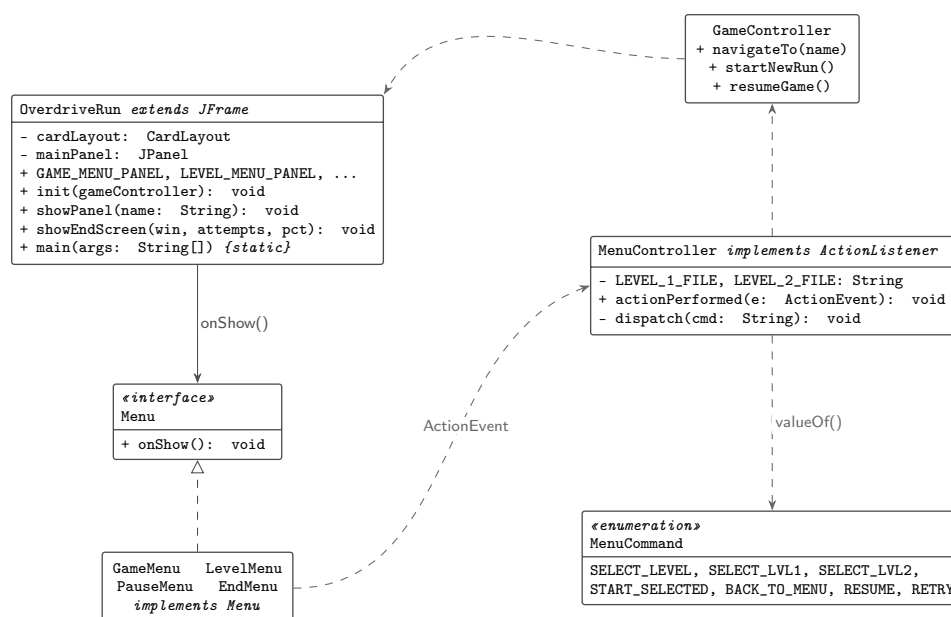


Figura 2.5: Navigazione fra le schermate e dispatch dei comandi di menu.

Soluzione. La navigazione è centralizzata in **OverdriveRun** tramite un **CardLayout**. Ogni pannello è registrato con una chiave stringa, definita come costante pubblica della classe frame, e il cambio di schermata avviene sempre tramite **showPanel()**, che oltre a mostrare il pannello invoca **onShow()** sull'oggetto **Menu** corrispondente, permettendo a ciascun pannello di aggiornarsi e di prendere il fuoco da tastiera prima di essere visualizzato.

I comandi dei pulsanti sono tipizzati tramite l'enum **MenuCommand**, che realizza il pattern Command. Ogni pulsante imposta il proprio action command con il nome del valore enum corrispondente, ad esempio **MenuCommand.RETRY.name()**. **MenuController**, che implementa **ActionListener**, riceve l'evento, converte la stringa nel valore enum tramite **MenuCommand.valueOf()** e fa dispatch in uno **switch**. Un comando non riconosciuto viene segnalato su **stderr** e ignorato, senza far cadere l'applicazione. L'approccio elimina le stringhe

magiche dal dispatch: un refactoring del nome del comando viene rilevato dal compilatore ovunque sia usato.

Tutte le azioni del `MenuController` sono schedulate tramite `SwingUtilities.invokeLater()` anche quando l'evento proviene già dall'EDT. Questo è necessario perché alcuni comandi, come `RETRY`, chiamano `stopLoop()`, che esegue una `join` sul thread di gioco: eseguirla direttamente nell'handler di un `ActionEvent` causerebbe un deadlock, poiché il thread di gioco tenta a sua volta di schedulare operazioni sull'EDT tramite `invokeLater`. Rimandare il dispatch lascia terminare l'evento corrente ed evita il blocco.

I metodi con cui il Controller serve la View sono divisi per responsabilità: `navigateTo()` cambia soltanto schermata, `startNewRun()` avvia un tentativo dall'inizio ed è il percorso condiviso da `PLAY` e `RETRY`. Tenere separate navigazione e ciclo di vita del tentativo evita che un cambio di pannello abbia effetti collaterali sul motore.

Due comandi sono raggiungibili anche da tastiera: `INVIO` nella schermata finale equivale a `RETRY`, `ESC` nel menu di pausa equivale a `CONTINUA`. Entrambi usano le key binding di Swing (`InputMap` e `ActionMap` con ambito `WHEN_IN_FOCUSED_WINDOW`) e non chiamano direttamente il Controller: costruiscono lo stesso `ActionEvent` che invierebbe il pulsante, così che scorciatoia e clic percorrano un'unica strada. È anche la ragione per cui `onShow()` richiede il fuoco da tastiera al pannello che sta per comparire.

La finestra, infine, disabilita la chiusura automatica e registra un `WindowAdapter` che ferma il ciclo di gioco prima di distruggere i pannelli, così che il thread di gioco non sopravviva alla superficie su cui invoca `repaint()`.

2.2.2.2 Gestione dell'audio e pattern Singleton

Problema. L'applicazione necessita di un unico gestore audio condiviso fra tutte le componenti: il Controller deve avviare e fermare la musica di sottofondo, mentre gli effetti sonori devono poter essere riprodotti in modo fire-and-forget da qualunque punto del codice, senza bloccare il thread chiamante.

Alternative considerate. Passare un'istanza del gestore audio a chi ne ha bisogno sarebbe stato più pulito e più facile da sostituire in un test, ma avrebbe richiesto di propagarla attraverso tre livelli fino a punti, come la riproduzione di un effetto sonoro, che non hanno altro motivo per conoscerla. Abbiamo considerato anche una classe di soli metodi statici, senza istanza: è la più sbrigativa, ma non lascia posto allo stato che serve davvero, cioè il clip aperto e la traccia in esecuzione. Il Singleton è un compromesso: rende globale una dipendenza, e questo è il suo difetto, ma qui la risorsa è effettivamente unica, la linea audio di sistema e il costo si limita a una classe difficile da sostituire in un test, cosa che non facciamo perché l'audio non è sottoposto a test automatici.

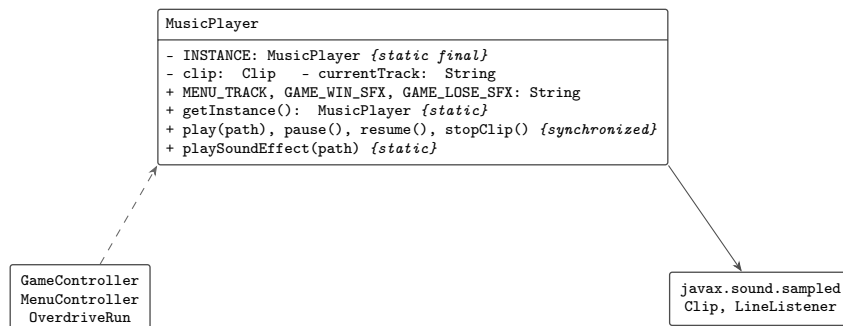


Figura 2.6: Gestione dell'audio.

Soluzione. La classe `MusicPlayer` è implementata come Singleton con inizializzazione eager: l'istanza unica è dichiarata come campo statico finale e creata al caricamento della classe, sfruttando la garanzia del class loader Java sulla thread-safety dell'inizializzazione statica, senza bisogno di `synchronized` né di `volatile`.

La musica di sottofondo è gestita tramite un `Clip` condiviso. I metodi `play()`, `pause()`, `resume()` e `stopClip()` sono tutti `synchronized` sull'istanza, così che due thread non possano modificare contemporaneamente lo stato del clip. Il metodo `play()` è idempotente: se il brano richiesto è già in esecuzione la chiamata è un no-op, evitando il riavvio percepibile della traccia quando si torna sulla stessa schermata.

Gli effetti sonori sono riprodotti tramite il metodo statico `playSoundEffect()`, che apre un `Clip` separato e indipendente su un thread demone dedicato. Un `LineListener` registrato sul clip lo chiude non appena la riproduzione termina, così la linea audio viene sempre rilasciata. Questo metodo non è `synchronized` per scelta progettuale: opera sempre su un clip distinto e non deve interferire con la musica di sottofondo né bloccare il chiamante, che è tipicamente l'EDT.

I percorsi delle risorse audio dell'applicazione, cioè la traccia dei menu e gli effetti di vittoria e sconfitta, sono raccolti in tre costanti pubbliche della classe, così che nessun pannello e nessun controller contenga il nome di un file sonoro. Restano fuori le tracce dei livelli, che sono un dato del Model: ciascun livello dichiara la propria nel JSON e `LevelLoader` conserva un percorso di riserva per il caso in cui il campo manchi.

2.2.2.3 Rendering e cache degli sprite

Problema. Il `GameRenderer` deve disegnare ogni frame a 60 Hz iterando su tutte le entità del livello e leggendone la sprite corrispondente. Caricare l'immagine dalla classpath a ogni frame sarebbe inaccettabile per le prestazioni. Occorre un meccanismo di caching trasparente e robusto di fronte a sprite mancanti o non decodificabili.

Alternative considerate. Caricare le immagini a ogni frame è stato escluso subito per il costo di I/O. Restava la scelta fra caricamento pigro puro, cioè decodificare la sprite alla prima richiesta, e precaricamento in costruzione: il primo distribuisce il costo e non paga per le sprite mai usate, ma concentra i cache-miss nei primi secondi di partita, cioè proprio dove uno scatto si nota. Con cinque sole immagini il precaricamento costa pochi millisecondi all'avvio e li elimina del tutto. Sul fallimento avevamo due strade: sollevare un'eccezione, che rende il problema impossibile da ignorare ma impedisce di giocare per un asset mancante, oppure il placeholder visibile, che lascia il gioco funzionante e rende comunque evidente l'errore. Abbiamo scelto la seconda, accettando che un asset mancante possa sfuggire se nessuno guarda lo schermo.

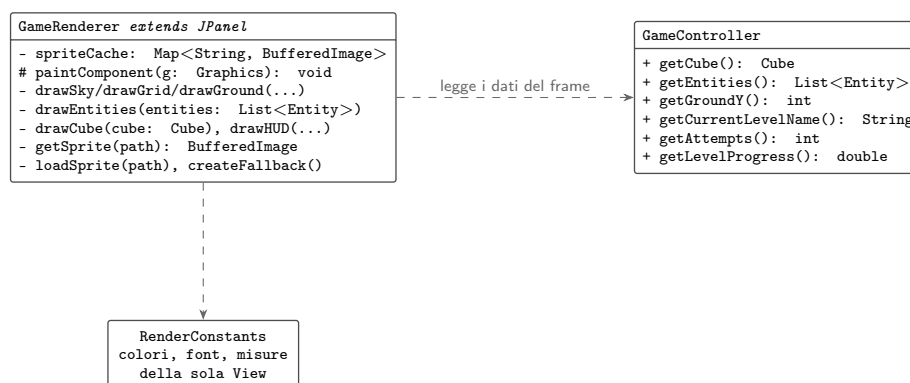


Figura 2.7: Rendering: il renderer dipende dal solo Controller.

Soluzione. Il renderer mantiene una `Map<String, BufferedImage>` indicizzata per percorso della risorsa. Alla costruzione del `GameRenderer` tutti gli sprite vengono precaricati, così che le richieste successive restituiscano il valore già memorizzato senza alcun accesso all'I/O e senza cache-miss iniziali durante la partita. L'elenco dei percorsi da precaricare è una costante del renderer: una nuova entità continua comunque a essere disegnata, perché la cache decodifica a richiesta, ma non beneficia del precaricamento finché quell'elenco non viene aggiornato.

La decodifica gestisce tre modi di fallire allo stesso modo: la risorsa non è presente sulla classpath, la lettura solleva un'eccezione, oppure `ImageIO.read()` restituisce `null`. Quest'ultimo caso è previsto dall'API quando lo stream esiste ma non contiene un formato immagine riconosciuto, ed è distinto dal lancio di un'eccezione: va quindi controllato esplicitamente, altrimenti nella cache non verrebbe memorizzato nulla e il tentativo di lettura si ripeterebbe a ogni frame. In tutti e tre i casi viene generata e memorizzata un'immagine magenta di dimensione standard, un colore che non compare in nessun asset reale ed è quindi immediatamente riconoscibile durante lo sviluppo, senza causare eccezioni a runtime.

Il metodo `paintComponent()` compone la scena in ordine *z* preciso: cielo, griglia, terreno, entità, cubo, HUD. Le entità sono disegnate *n* volte affiancate orizzontalmente a partire dalla posizione del `Pointer`, dove *n* è il campo `number` definito nel JSON. La quota del terreno non è una costante della View ma viene letta dal Controller, così che un livello con un `ground_y` diverso resti disegnato correttamente.

L'HUD include una barra di avanzamento il cui colore è interpolato linearmente fra blu e verde in base alla progressione del cubo nel livello, calcolata frame per frame, e la percentuale corrispondente.

Capitolo 3

Sviluppo

3.1 Testing automatizzato

I test sono sviluppati con JUnit 5 e organizzati in classi tematiche nel package `test`, ciascuna dedicata a un aspetto della logica implementata nel Model. Complessivamente la suite conta 43 metodi di test, che danno luogo a 48 esecuzioni per effetto dei test parametrizzati.

Impostazione. Tutti i test che riguardano il comportamento del gioco sono eseguiti contro il `GameEngine` reale, non contro una sua riproduzione. Per rendere questo possibile senza dipendere dal file system, il motore espone un secondo costruttore che riceve la sorgente delle entità e la quota del terreno:

```
public GameEngine(Supplier<List<Entity>> entityFactory, int groundY)
```

Il percorso normale, basato su file, usa esattamente lo stesso campo (`loader::getMap`), quindi non esistono due strade diverse fra produzione e collaudo. La classe di supporto `TestLevels` è una fixture di poche righe che costruisce un motore su un livello sintetico e offre due utilità: avanzare di un numero dato di tick e misurare l'apice raggiunto dal cubo.

La suite è eseguita da Maven tramite il plugin Surefire, quindi non dipende dalla configurazione dell'IDE.

Determinismo. Il motore è completamente deterministico: gravità, impulso di salto e velocità di scorrimento sono costanti intere o decimali fisse, e non c'è alcuna sorgente di casualità. Le asserzioni possono quindi essere esatte anziché approssimate. Con `GRAVITY` pari a 0.8 e `JUMP_POWER` pari a -12, un salto dal terreno raggiunge l'apice a $y = 284$, cioè 76 pixel sopra il suolo, e atterra al trentesimo tick: sono questi i valori che i test verificano. Anche le posizioni delle entità negli scenari di collisione sono scelte con lo stesso criterio, in modo che il cubo le incontri in un punto preciso del proprio arco: il blocco su cui deve atterrare, per esempio, è collocato in modo da essere raggiunto in fase discendente, perché in fase ascendente l'urto sarebbe correttamente letale.

Classi di test.

- **HitboxTest** verifica in isolamento la primitiva geometrica su cui si decide ogni collisione: memorizzazione di posizione e dimensioni, traslazione cumulativa con conservazione delle dimensioni, `setPosition()`, simmetria e correttezza di `intersects()` compresi i casi limite di rettangoli contenuti e di rettangoli adiacenti che si toccano soltanto lungo un bordo, e natura difensiva della copia restituita da `getBounds()`.

- **EntityTest** verifica la gerarchia delle entità in tre gruppi annidati: geometria delle hitbox per ciascun tipo, con test parametrizzati sul numero di tile n ; comportamento di `scroll()`, che deve muovere posizione visiva e hitbox dello stesso delta, non alterare mai la coordinata verticale e preservare la spaziatura relativa fra entità distinte; effetto restituito da `onCollision()` per ciascun tipo, incluso il caso non banale di `MultipleBlock`, che deve distinguere atterraggio dall'alto, impatto laterale e urto dal basso.
- **PhysicsTest** verifica il moto del cubo su un livello vuoto: posizione iniziale, assenza di attraversamento del terreno, arco di salto completo con apice e tick di atterraggio esatti, impossibilità del doppio salto quando il cubo è in aria anche premendo a ogni tick, ripetibilità del salto successivo, e le due condizioni del jump buffer, una richiesta entro la finestra viene applicata all'atterraggio, una richiesta troppo anticipata scade e viene scartata.
- **CollisionTest** verifica la reazione del motore a ciascun tipo di entità: game over al contatto con uno spike e assenza di falsi positivi a distanza; letalità dell'urto laterale contro un blocco e atterraggio corretto sulla sua superficie superiore, con il cubo che resta esattamente appoggiato senza compenetrazione; impulso automatico del `LaunchPad`, confrontato con l'altezza di un salto normale; comportamento condizionale del `JumpOrb`, inerte in assenza di una richiesta pendente e più potente di un salto normale in sua presenza; completamento del livello al contatto con il traguardo e assenza di completamento quando è ancora lontano.
- **GameStateTest** verifica lo stato che il motore espone al resto dell'applicazione: valori iniziali dei flag e del contatore dei tentativi, immodificabilità della lista di entità restituita alla View, natura di copia dell'oggetto restituito da `getCube()`, congelamento dello stato dopo un game over, conservazione del contatore dei tentativi attraverso un reset, e calcolo della progressione, che dopo 50 tick su un traguardo posto a 800 pixel di distanza deve valere esattamente 0.5.

La suddivisione per responsabilità, geometria, entità, fisica, collisioni, stato di gioco — rispecchia la stessa separazione dei concern adottata nel Model, e l'uso di `TestLevels` come fixture condivisa permette a ciascun test di partire da uno stato noto e riproducibile.

Copertura. La suite copre la logica di gioco, geometria, entità, fisica, collisioni e stato — ma non il caricamento dei livelli: `LevelLoader` e `ObstacleGenerator` non sono esercitati da alcun test automatico. È lo scoperto più significativo della suite, perché sono le due classi in cui un errore si manifesta a runtime anziché in compilazione: il parsing del JSON e la risoluzione riflessiva del costruttore. La verifica di quel percorso è avvenuta soltanto per via manuale, caricando i due livelli inclusi e introducendo di proposito segmenti malformati per osservare che venissero segnalati e saltati.

3.2 Metodologia di lavoro

L'impegno complessivo è stimabile in circa 35 ore a testa, di cui poco meno della metà spese in analisi e progettazione prima di scrivere codice.

Il progetto è stato sviluppato in coppia con un approccio top-down: prima analisi e modellazione del dominio, poi progettazione dell'architettura MVC, infine implementazione e testing. Il codice è stato versionato con Git su un repository condiviso su GitHub; ogni funzionalità è stata sviluppata su un branch dedicato e integrata tramite pull request, così da poter rivedere le modifiche dell'altro prima di unirle al ramo principale. La comunicazione è avvenuta sia in presenza sia tramite messaggistica istantanea, per accordarsi sulle interfacce prima di implementarle separatamente.

Strumenti utilizzati.

- **Git** e **GitHub**: versionamento, sviluppo cooperativo e code review.
- **Eclipse**: ambiente di sviluppo principale.
- **Maven**: gestione delle dipendenze (JSON-java per il parsing dei livelli, JUnit 5 per i test) e build del progetto. Il plugin Surefire rende la suite eseguibile con `mvn test`, sia dall'IDE sia da riga di comando.
- **JUnit 5**: framework per il testing automatizzato.
- **draw.io** e **StarUML**: prime stesure dei diagrammi.
- **Overleaf**: redazione della presente relazione in $\text{\LaTeX}$; i diagrammi di questa versione sono disegnati direttamente in TikZ, così che restino allineati al codice e modificabili insieme al testo.

3.2.1 Alessio Cavalieri

Componenti sviluppate in autonomia:

- package model: `GameEngine`, `Entity`, `AbstractEntity`, `Hitbox`, `Pointer`, `Effect`, `GameConstants`;
- tutte le entità di gioco: `MultipleSpike`, `MultipleBlock`, `LaunchPad`, `JumpOrb`, `EndLevel`;
- `LevelLoader`: parsing JSON e conversione delle coordinate logiche in pixel, con l'I/O isolato in un metodo separato rispetto al parsing;
- `ObstacleGenerator`: factory riflessiva con cache dei costruttori;
- logica di fisica e collisioni in `GameEngine`: gravità, jump buffer, rilevamento delle collisioni e applicazione degli `Effect`.

3.2.2 Alessandro Catalano

Componenti sviluppate in autonomia:

- package controller: `GameController` con il ciclo di gioco e la gestione del thread, `MenuController`, `LevelController`, enum `MenuCommand`;
- package view: `OverdriveRun` con `CardLayout` e navigazione, `GameRenderer` con il rendering completo della scena, tutti i pannelli di menu, `MenuStyle`, `RenderConstants`;
- package service: `MusicPlayer`, con musica di sottofondo ed effetti sonori su thread separati;
- HUD: nome del livello, contatore dei tentativi, barra di avanzamento con colore interpolato;
- cache degli sprite con precaricamento e immagine di fallback.

3.2.3 Componenti sviluppate in collaborazione

- definizione dell'interfaccia `Entity` e del contratto di `onCollision()`;
- struttura del file JSON dei livelli e convenzioni di naming;
- integrazione del ciclo di gioco con il pannello di rendering;
- separazione delle costanti condivise fra Model e View (`GameConstants` e `RenderConstants`);
- protocollo di comunicazione Controller $\rightarrow$ View tramite i metodi pubblici di `GameController`;
- progettazione del sistema di pausa e ripresa senza perdita di stato.

3.3 Note di sviluppo

Questa sezione riporta, separatamente per ciascun membro del gruppo, le feature avanzate del linguaggio utilizzate nel progetto e gli algoritmi di particolare interesse.

3.3.1 Alessio Cavalieri

Feature avanzate del linguaggio utilizzate.

- **Stream** e **Optional** in `GameEngine.resetGame()`: la catena `stream().filter(...).mapToInt(...).findFirst()` individua l'entità traguardo e ne memorizza la posizione iniziale, risolta con `orElse(-1)` quando il livello non ne contiene.
- **Stream** e **Optional** in `GameEngine.getLevelProgress()`: la catena `filter(...).findFirst().map(...)` produce un `Optional`, risolto con `orElse(0.0)`: sostituisce un controllo esplicito sul valore nullo.
- **Method reference** `loader::getMap` come `Supplier<List<Entity>`: la sorgente delle entità di un livello è una funzione, non un oggetto concreto, e questo rende il motore indipendente dal caricamento da file.
- **Method reference** a metodo d'istanza di un oggetto particolare: `EndLevel.class::isInstance` lega la chiamata all'oggetto `Class` di `EndLevel` ed è usato come predicato di `filter`.
- **Reflection** in `ObstacleGenerator`: `Class.forName()` e `getConstructor(...)` risolvono classe e costruttore a partire dal nome letto nel JSON.
- **Costrutti di concorrenza**: `synchronized` su tutti i metodi di `GameEngine` che accedono a stato mutabile condiviso.
- **Record** in `LevelLoader`: il tipo `LevelData` raccoglie i quattro campi di testata del livello ed è il valore restituito da `parseJson()`. Essendo immutabile, con costruttore e accessori generati dal compilatore, evita sia una classe di soli campi sia il ritorno di più valori tramite parametri di uscita.

Algoritmo di interesse: jump buffering. Il sistema accetta un input di salto fino a `JUMP_BUFFER_FRAMES` tick prima che il cubo tocchi terra, permettendo al giocatore di premere in anticipo e ottenere comunque il salto. La richiesta è rappresentata da un flag e da un contatore incrementato a ogni tick: il flag viene consumato all'atterraggio o scartato quando il contatore supera la finestra, senza alcuna struttura dati aggiuntiva. È un accorgimento che non appartiene ad alcuna libreria e che incide in modo netto sulla sensazione di controllo del gioco.

3.3.2 Alessandro Catalano

Feature avanzate del linguaggio utilizzate.

- **Lambda expressions** in `SwingUtilities.invokeLater(() -> ...)`, nel ciclo di gioco e in `pauseGame()`, per schedulare le transizioni di pannello sull'EDT.
- **Lambda expressions** in `sfx.addLineListener(e -> ...)` in `MusicPlayer.playSoundEffect()`, per chiudere il clip al termine della riproduzione.
- **Stream** in `OverdriveRun.findPanel()`: `Arrays.stream(...).filter(...).findFirst()` localizza il pannello corrente, sfruttando la valutazione short-circuit di `findFirst()` per fermarsi al primo risultato utile.

- **Method reference** `this::loadSprite` passato a `computeIfAbsent` in `GameRenderer.getSprite()`: la decodifica avviene solo alla prima richiesta di ciascuna risorsa.
- **Method reference** `this::handleGameOver` passato a `invokeLater` al termine di un tentativo.
- **Pattern matching per instanceof** in `showPanel()`: `if (target instanceof Menu menu) menu.onShow();` unisce verifica del tipo e dichiarazione della variabile.

Algoritmo di interesse: fixed-timestep loop con catch-up. Descritto in dettaglio nella sezione 2.2.1.3: il ciclo mantiene 60 aggiornamenti logici al secondo stabili, indipendentemente dalla durata del rendering, recuperando il ritardo con tick consecutivi e senza ricorrere al busy-waiting.

Nota sui generici. Il progetto non definisce tipi generici propri né generici con vincoli: usa soltanto classi generiche di libreria `List<Entity>`, `Map<String, BufferedImage>` e `Supplier<List<Entity>>` che non costituiscono di per sé un uso avanzato. Preferiamo dichiararlo esplicitamente piuttosto che presentarlo come tale.

Conclusioni

Risultati ottenuti

Tutti i requisiti obbligatori sono stati implementati e sono verificabili eseguendo il gioco: due livelli selezionabili, game loop strutturato a 60 Hz, hitbox e collisioni affidabili, salto a velocità di scorrimento costante, entità che alterano la traiettoria verticale e sistema di pausa. Sono state realizzate anche tutte le funzionalità opzionali dichiarate in fase di analisi, colonna sonora compresa.

L'architettura ha retto alla prova più significativa, cioè l'aggiunta di nuove entità: introdurre un tipo di ostacolo richiede una sola classe nel package `model` e una riga nel file di livello, senza toccare il motore, il renderer o il controller.

La suite di test conta 43 metodi e 48 esecuzioni, tutte verdi, ed esercita il motore reale e non una sua riproduzione.

Limiti noti

- I livelli disponibili sono due e sono scritti a mano nei file JSON: non esiste un editor, e la stesura di un nuovo livello richiede di calcolare le coordinate in tile.
- Non c'è persistenza: il contatore dei tentativi si azzerà alla chiusura dell'applicazione e non viene salvato alcun risultato.
- Il rendering non interpola fra due tick logici. A 60 Hz la differenza non è percepibile, ma su un monitor a frequenza più alta il movimento non risulterebbe più fluido di così.
- La lista di entità che il renderer riceve è una vista non modificabile sulla lista viva: le entità continuano a essere spostate dal thread di gioco durante il disegno. La scelta è deliberata, perché costruire una copia coerente a ogni frame avrebbe un costo non giustificato dal beneficio, ma significa che un frame può teoricamente mostrare due entità aggiornate in tick diversi.
- Il caricamento dei livelli non è coperto da test automatici. La separazione fra I/O e parsing in `LevelLoader` era stata pensata anche in questa direzione, ma `parseJson()` è package-private nel package `model` e le classi di test vivono nel package `test`, quindi non lo raggiungono. Spostare i test nel package del modello renderebbe la verifica immediata.
- La fila di spike condivide un'unica area di collisione, rientrata di 5 pixel solo alle due estremità. Con `n` elevato (nel secondo livello si arriva a undici tile) la regione letale copre anche gli avvallamenti fra una punta e la successiva, dove graficamente non c'è nulla. Servirebbe una hitbox per tile anziché una per segmento.
- L'area di collisione del `JumpOrb` è sempre di un solo tile e ignora il campo `n`, mentre il renderer disegna comunque `n` sprite affiancati. Nei livelli inclusi `n` vale sempre 1, quindi l'incoerenza non è osservabile, ma un orb ripetuto mostrerebbe sprite privi di effetto.

- Il precaricamento degli sprite si basa su un elenco di percorsi scritto a mano in `GameRenderer`. Una nuova entità continua a funzionare, perché la cache decodifica comunque a richiesta, ma non beneficia del precaricamento finché l'elenco non viene aggiornato.
- La ripetizione del salto a tasto premuto si appoggia all'auto-repeat della tastiera fornito dal sistema operativo, il cui ritardo iniziale dipende dalla configurazione della macchina. Tracciare esplicitamente lo stato del tasto in `keyReleased()` renderebbe il comportamento deterministico quanto il resto del motore.
- La finestra ha dimensioni fisse e il gioco non è ridimensionabile.

Sviluppi futuri

Gli sviluppi più naturali sono un editor di livelli che produca direttamente il JSON, il salvataggio dei risultati su file, e nuove entità di gioco (piattaforme mobili, ostacoli a comparsa) che l'attuale gerarchia consentirebbe di aggiungere senza modifiche al motore. Sul fronte della qualità, l'intervento più utile sarebbe l'estensione dei test al caricamento dei livelli, oggi l'unica parte della logica priva di collaudo automatico.